ReqFlow AI
Final Capstone Project Report

Agentic Requirements-to-User-Story Automation · Finalized use case #83

Synthetic Student Attendance Management System · 10 November 2026

Submitted as a functional capstone proof of concept; partial backlog sign-off explicitly disclosed.

Acceptance evidence	Observed result
Python regression	58 tests passed
Authenticated smoke	8/8 checks passed
Delivery artifact audit	passed: true
Delivery output	Design and plan approved; 12 tasks
Eligible stories	4 quality-cleared + PO-approved of 10
MCP	3 tools discovered; preview dry-run; no writes

Executive summary

ReqFlow AI ingests a synthetic BRD, retrieves source evidence using pgvector, generates requirements and user stories using LangGraph/Ollama, applies quality and PO approval gates, synthesizes a solution design and delivery plan, and exposes GitHub/Jira publishing via controlled APIs and a demonstrated MCP client/server. The final result is a functioning proof of concept, not an implementation of the attendance product.

1. Use case, boundaries and requirements

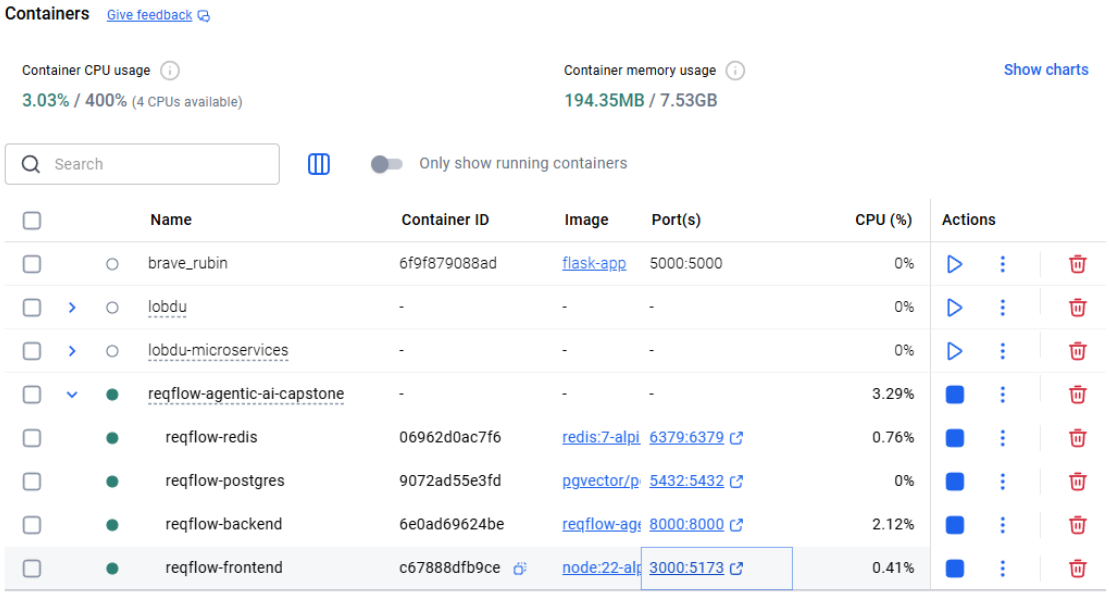
Finalized use case: Requirements to User Story Automation. Required capabilities: business requirements extraction, stories and acceptance criteria, solution-design documents, implementation tasks, and estimates. Conceptual roles: Research, Document Creation and Planning Agents. The APIs/MCP reference catalog informed integration choices but was not an obligation to integrate every service.

The sample BRD is synthetic and contains one business objective (BRD-ATT-001), five FRs (FR-001–005) and four NFRs (NFR-001–004). Its NFR-004 has no approved numeric percentile/response-time/load target. FR-005 correction authority also needs policy clarification.

2. Architecture and technology

```
React / Vite -> FastAPI (sessions, RBAC, APIs)
  -> PostgreSQL + pgvector (source, versions, decisions, audit)
  -> LangGraph + Ollama (analysis/story workflows)
  -> Deterministic design and planning nodes (Phase 7)
  -> Publishing service -> GitHub / Jira
MCP stdio client <-> MCP publishing server (read-only preview)
```

Docker Compose services: frontend, backend, Ollama, PostgreSQL and Redis. Models: qwen2.5:3b for generation and nomic-embed-text for 768-dimensional embeddings. Phase 7 design output is a proposed attendance application architecture (UI → API → services → database → audit/observability), not implemented attendance code.



Evidence 01 — Docker Compose containers (local screenshot).

3. Agent-to-agent workflow

BRD upload -> RAG / Research -> Source-backed requirements
-> LangGraph story generation + quality gate -> PO story approval
-> Phase 7 solution design -> PO design approval
-> Implementation task plan -> PO plan approval

- Research Agent: parse, embed, retrieve and source-ground requirements; surface quality findings and open questions.
- Story Creation Agent: create epics, stories and Given–When–Then AC; validate, repair and retain versions.
- Solution Design Agent: deterministic design proposal with components, APIs, entities, assumptions, traceability and open decisions.
- Implementation Planning Agent: design/implementation/testing tasks, dependencies, priority, provisional estimates and refinement flags.
- Unified orchestration: LangGraph downstream replay of saved, verified outputs, with independent human gates; no autonomous approval or provider publishing.

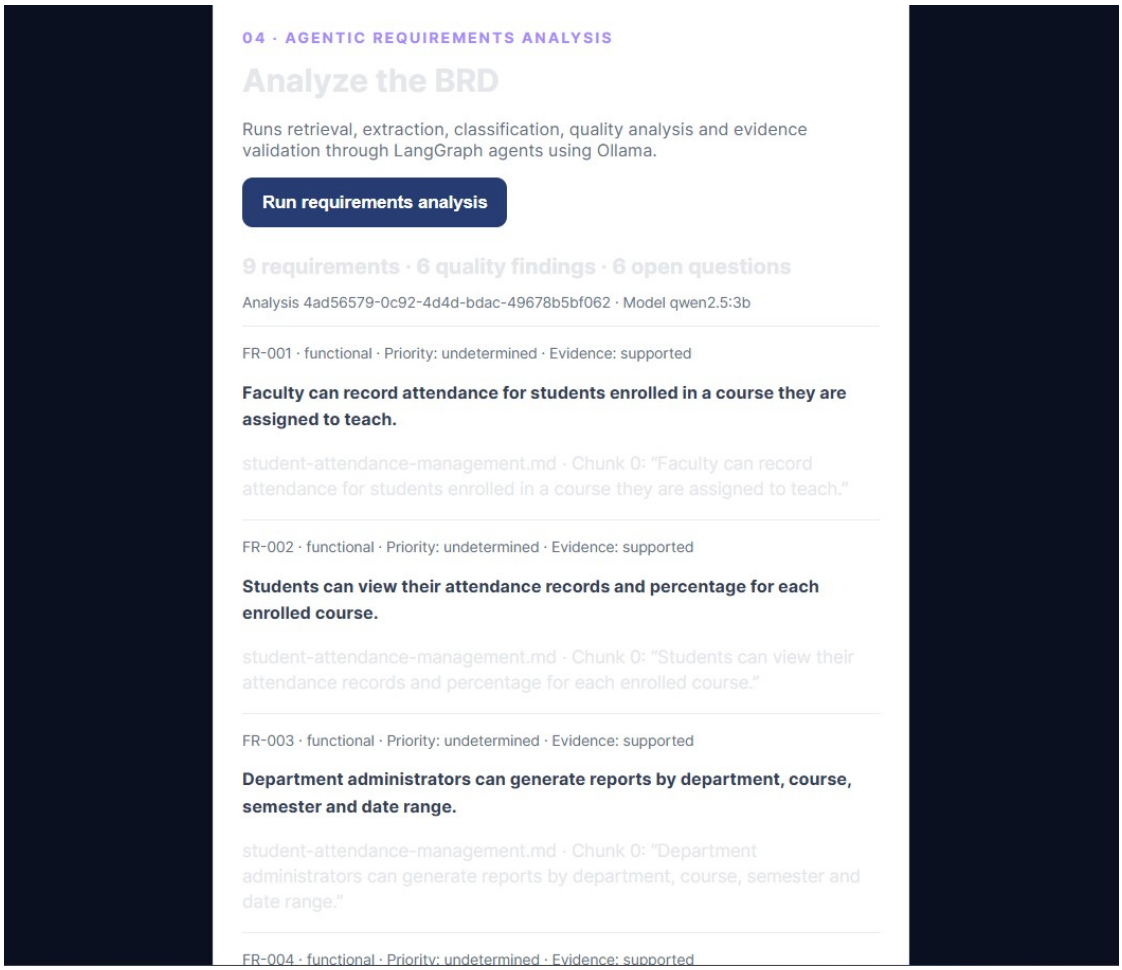
4. Seven implementation phases

Phase	Implemented capability	Validation
1	Infrastructure, database and frontend	Containers/health
2	BRD ingestion, chunking, embeddings, RAG	Indexed document, semantic search
3	Source-backed requirements analysis	Requirements + evidence

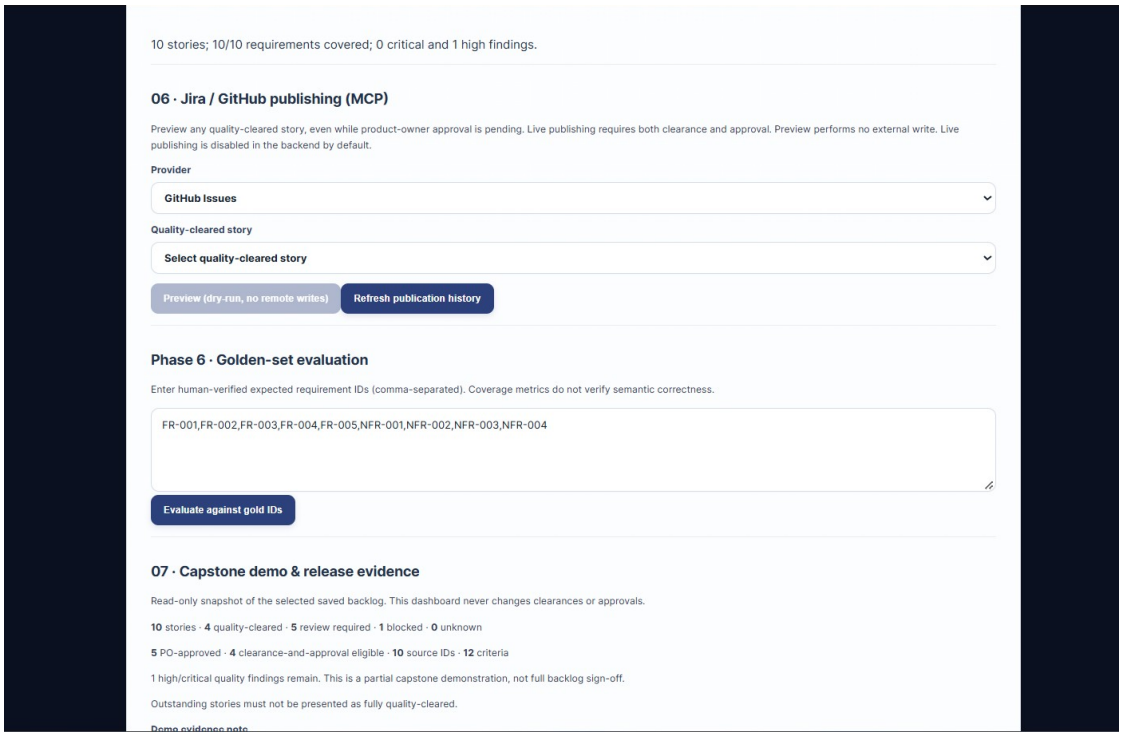
4	Backlog, AC, repair, quality gates	10 stories; 4 cleared
5	GitHub/Jira service + MCP server	Provider issues + MCP demo
6 / 6.1	RBAC, evaluation, audit, PO clarifications	Approval and category metrics
7	Design, planning, orchestration, read-only evidence	Saved approved artifacts; tests

5. Example requirements and generated stories

The ten-ID golden set (including BRD-ATT-001) matched 10/10 requirement identifiers (precision, recall and F1 = 1.00). The nine-ID FR/NFR-only reference set produced 94.74% F1 because it omitted the predicted business objective. All nine FR/NFR implementation IDs were covered; these are coverage metrics rather than semantic-correctness measures.



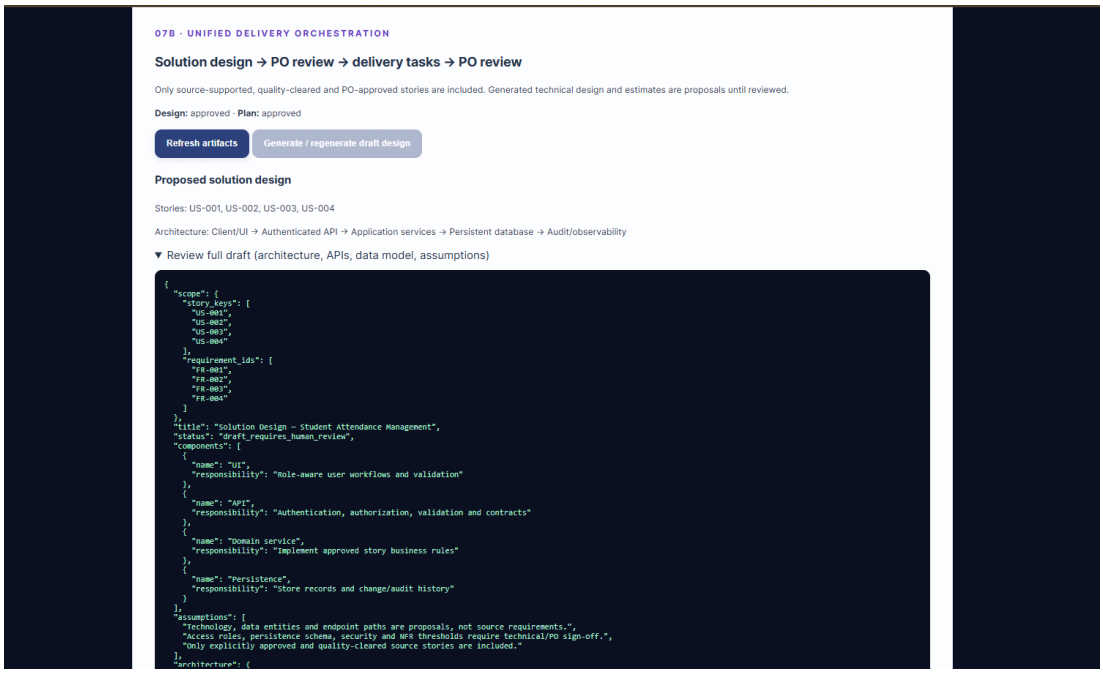
Evidence 07 — Requirements analysis output (source-grounded).



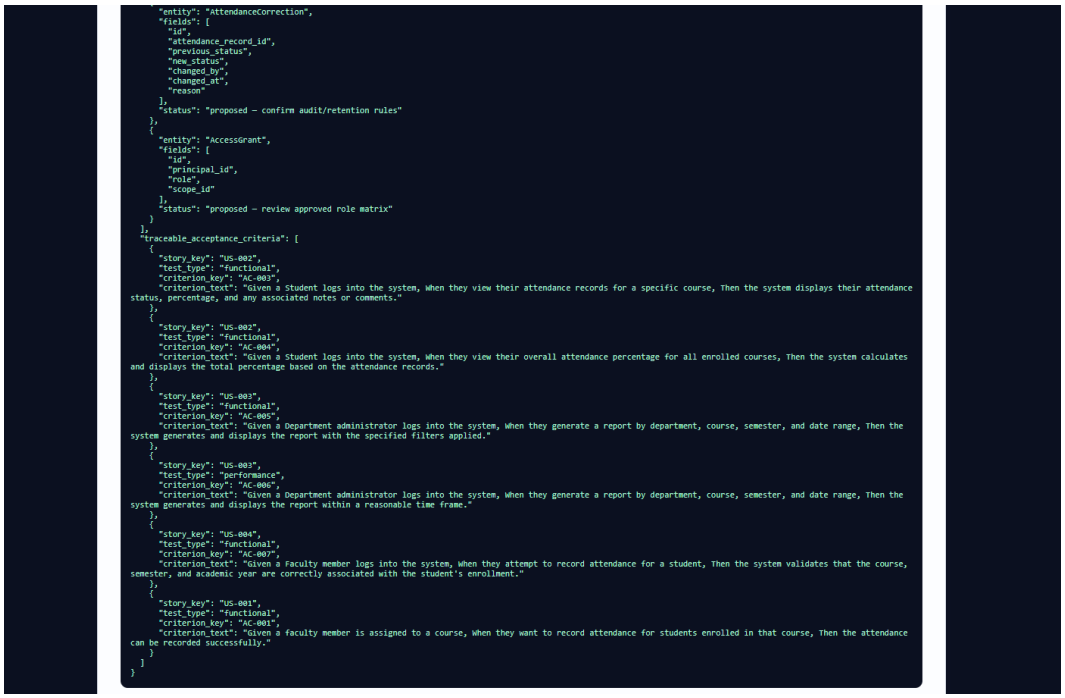
Evidence 08 — Saved epic/story/acceptance-criteria backlog.

6. Solution Design Agent — persisted result

Saved design status: approved. Scoped to US-001–US-004 (FR-001–FR-004). The artifact includes proposed UI/API/domain/persistence responsibilities, candidate API contracts, data entities, traceability, assumptions and open decisions. Proposals require technical review and are not production implementations.



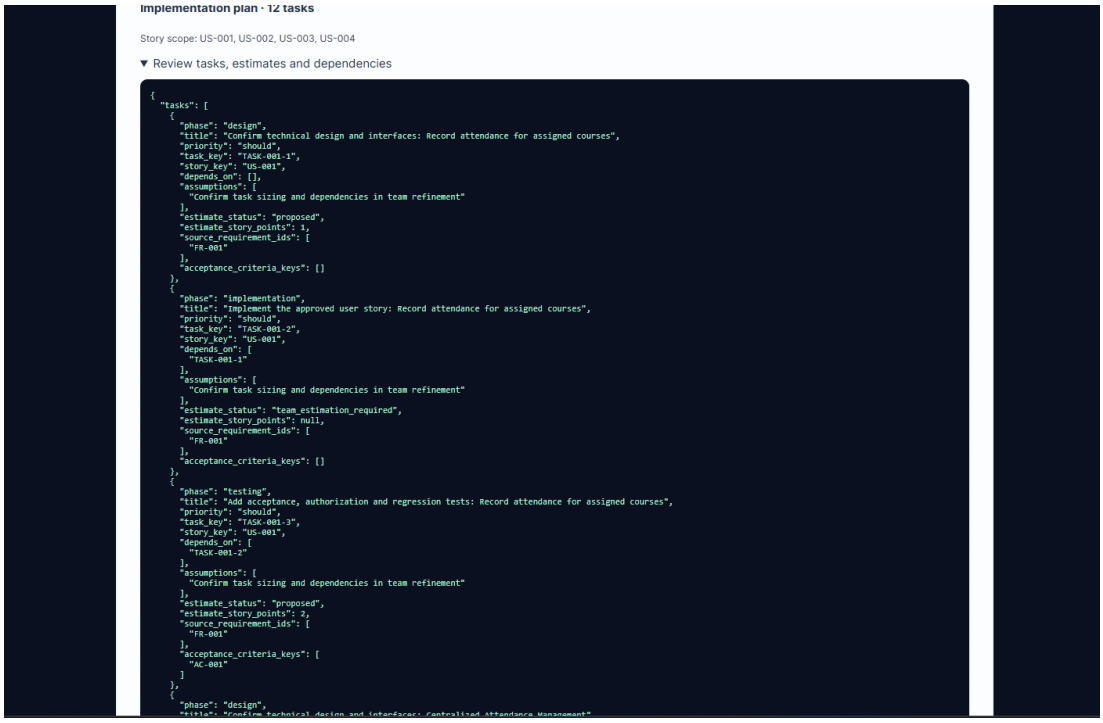
Evidence 12 — Design approval and persisted artifact overview.



Evidence 14 — Proposed architecture/APIs/entities and traceability (excerpt).

7. Implementation Planning Agent — persisted result

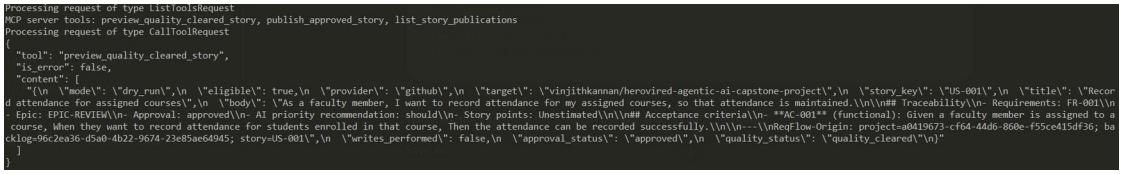
Saved plan status: approved; 12 tasks (4 design, 4 implementation, 4 testing), 8 dependency edges, unique task keys and valid dependency references. Two tasks require team estimation; all estimates are provisional.



Evidence 15 — Implementation-plan JSON and generated tasks.

8. MCP and external integrations

MCP stdio tools discovered: preview_quality_cleared_story, publish_approved_story, list_story_publications. The real client invoked preview_quality_cleared_story for US-001 and received is_error=false, eligible=true, approval_status=approved, quality_status=quality_cleared and writes_performed=false. The publish tool was not invoked in this MCP demonstration.

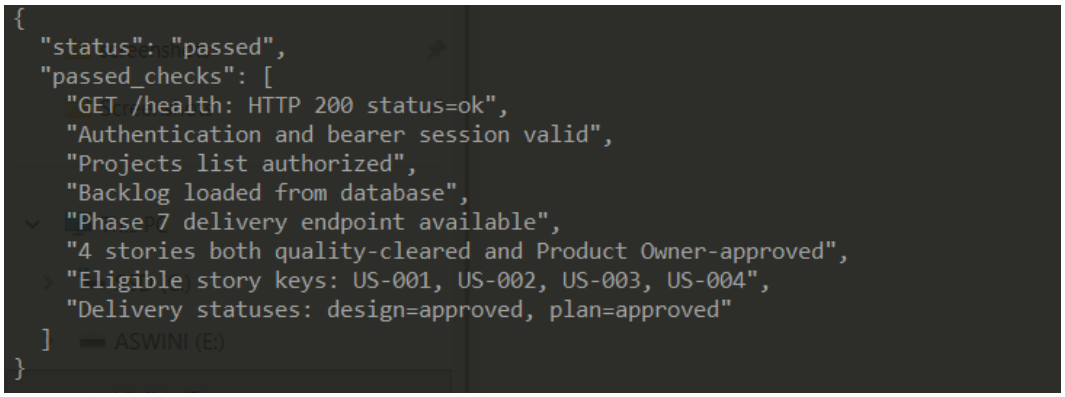


Evidence 20 — Actual MCP tool invocation; dry-run, no provider writes.

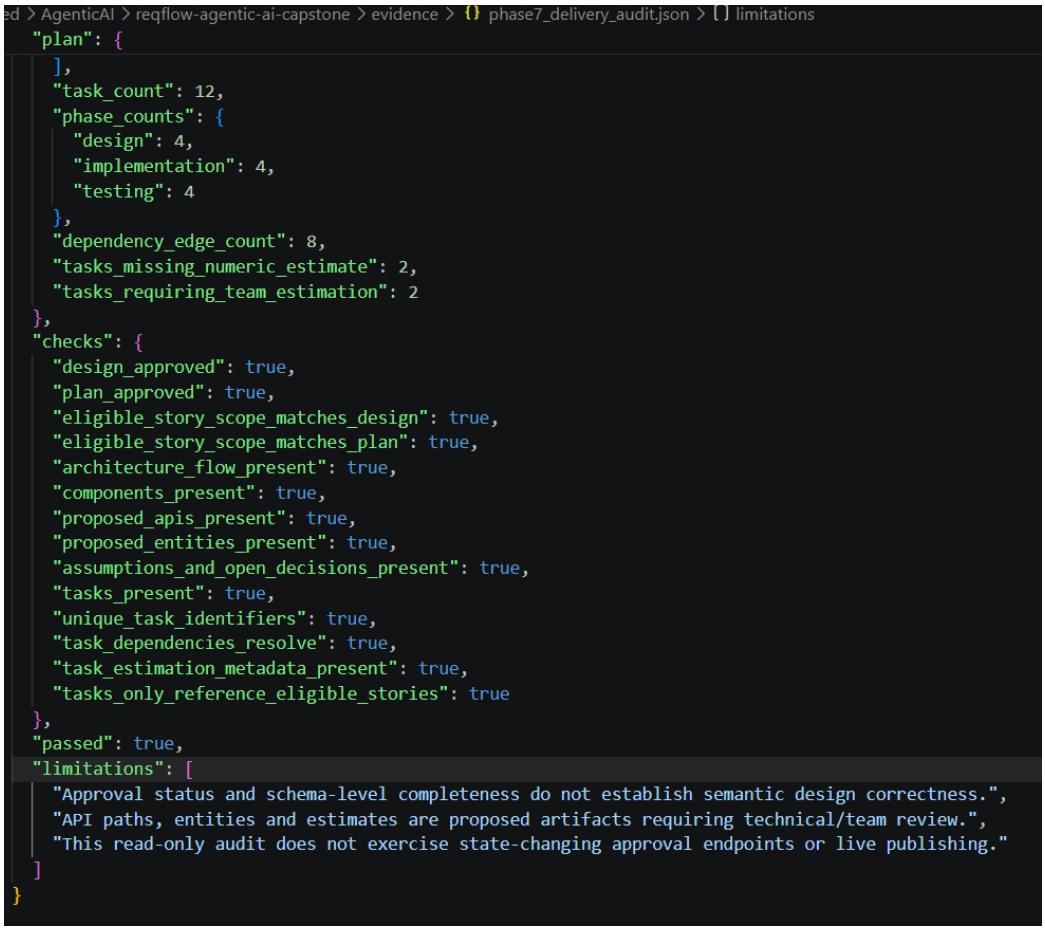
Separate earlier evidence shows successful GitHub and Jira publications through the application's provider integration, with approvals, origin traceability and duplicate-prevention controls.

9. Validation and measured outcomes

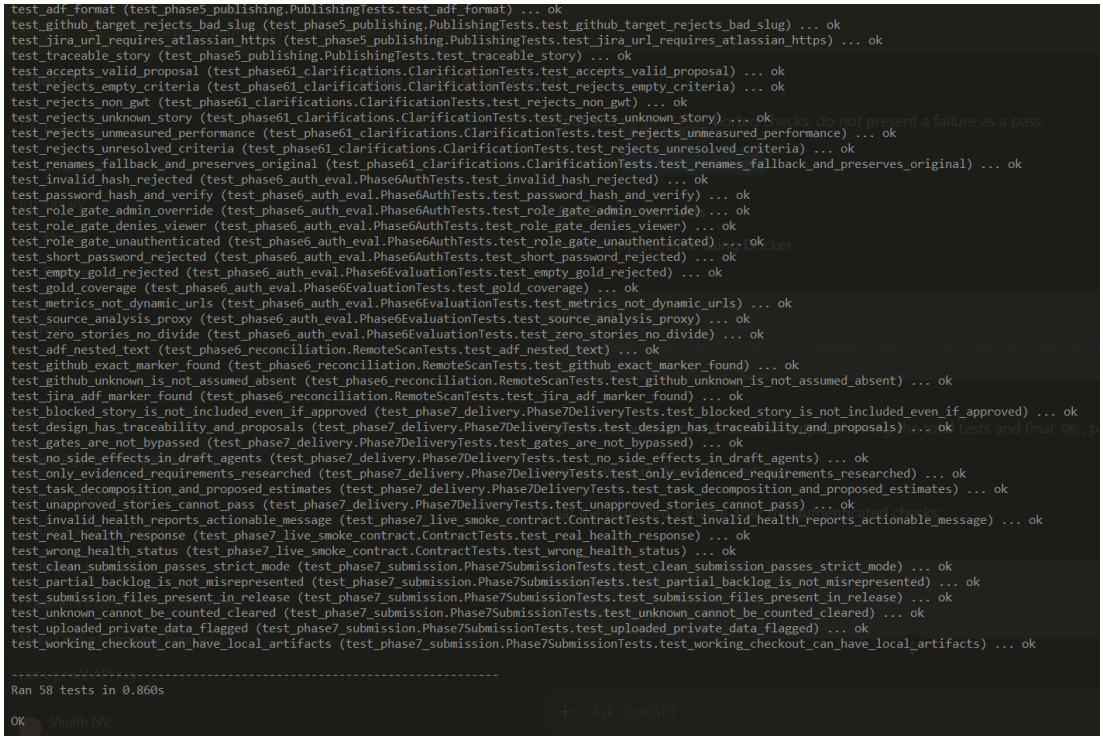
Check	Outcome	Interpretation
Docker Python tests	58 / 58 passed	Unit/contract checks
Authenticated smoke	8 / 8 passed	API, auth, DB-backed saved artifacts
Artifact-content audit	passed: true	Content/schema checks, not semantic sign-off
Eligible stories	US-001–US-004	4 of 10 quality-cleared and approved
Plan	12 tasks / 8 dependencies	2 numeric estimates pending
MCP	Discovery and preview passed	No remote write



Evidence 21 — Eight live smoke checks passed.



Evidence 22 — Audit check set and overall passed=true.



Evidence 23 — Full Python test run: 58 tests, OK.

10. Security, reproducibility and reviewer guidance

Authentication uses revocable hashed bearer sessions and role checks (admin/product_owner/engineer/viewer). Quality clearance is distinct from PO approval. Migrations 001–009 cover document/RAG, analysis, backlog, publishing, security and delivery. Full source and synthetic BRD are included; live PostgreSQL records and model binaries are not.

```
docker compose config --quiet
docker compose up -d --build
docker compose ps
# UI: localhost:3000 ; Swagger: localhost:8000/docs
# Do NOT use docker compose down -v on a working installation
```

11. Limitations, residual work and ethical reporting

- Only four of ten generated stories are quality-cleared and PO-approved. Five require review; one is blocked by missing NFR-004 performance acceptance targets.
- Clarification decisions were approved, but a fully finalized and quality-cleared new backlog version was not demonstrated.
- The approved design is a proposed architecture; not implemented attendance-product functionality or semantic design validation.
- Implementation tasks need technical refinement; two numeric estimates remain unset.
- The MCP client preview was read-only. It does not establish LLM-directed autonomous tool selection or live publishing in that particular demo.
- No production security, real-world SLA/load test, or full external-provider integration certification is claimed.

12. Evidence index and submission contents

The complete evidence gallery contains 23 normalized screenshots in data/screenshots/, including infrastructure, ingestion, retrieval, requirements, backlog, GitHub/Jira publishing, design, planning, MCP, smoke test, audit and unit tests. Their source is real user-supplied local captures. See docs/FINAL_CAPSTONE_REPORT.md for a clickable evidence index, commands and full disclosure.

Package includes source, docs, database migrations, synthetic input BRD, test and smoke scripts, final report (Markdown/Word/PDF), and data/screenshots. Secrets, runtime document uploads, database dumps and local node_modules are excluded.